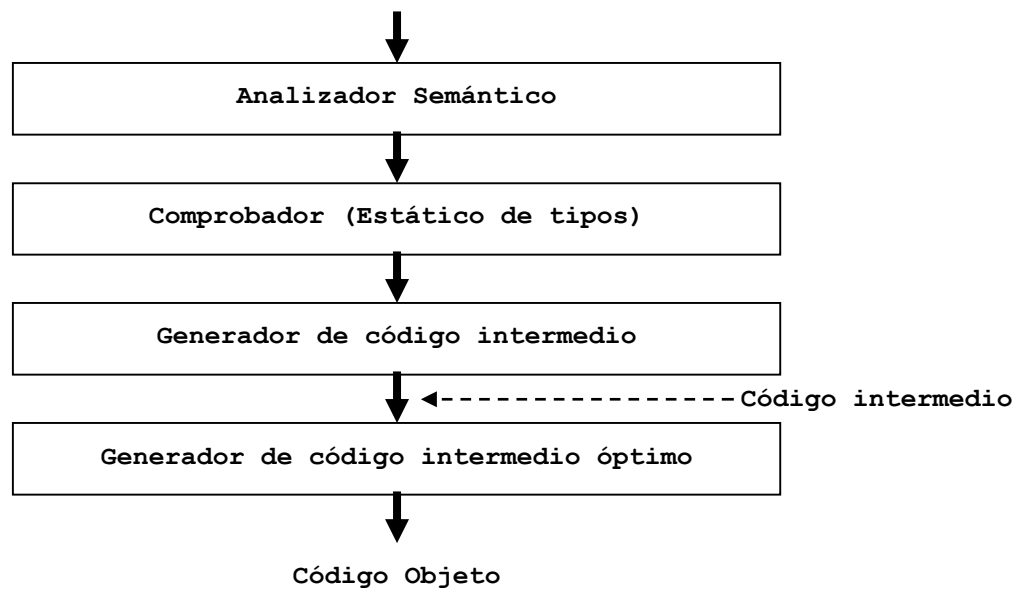


UNIDAD III

Generación de Código Intermedio

3.1 Introducción

Un compilador en su etapa inicial traduce un programa fuente a una representación intermedia a partir de la cual la etapa final genera el código objeto.



3.2 Códigos Intermedios

Códigos Intermedios:

- ❖ Árbol Sintáctico.
- ❖ Notación Postfija.
- ❖ Código de 3 direcciones.

Código de 3 direcciones (C3D) .

Recibe este nombre porque participan al máximo 3 variables y normalmente utiliza un operador.

La secuencia de producciones tiene la siguiente forma general:

$$x := y \text{ op } z$$

donde:

x, y, z Son nombres, variables, constantes, o variables temporales

op Es cualquier operador lógico o aritmético.

Ejemplo:

- ❖ La expresión $x + y * z$ se traduce a C3D como:

$$\begin{aligned} t1 &:= y * z \\ t2 &:= x + t1 \end{aligned}$$

donde:

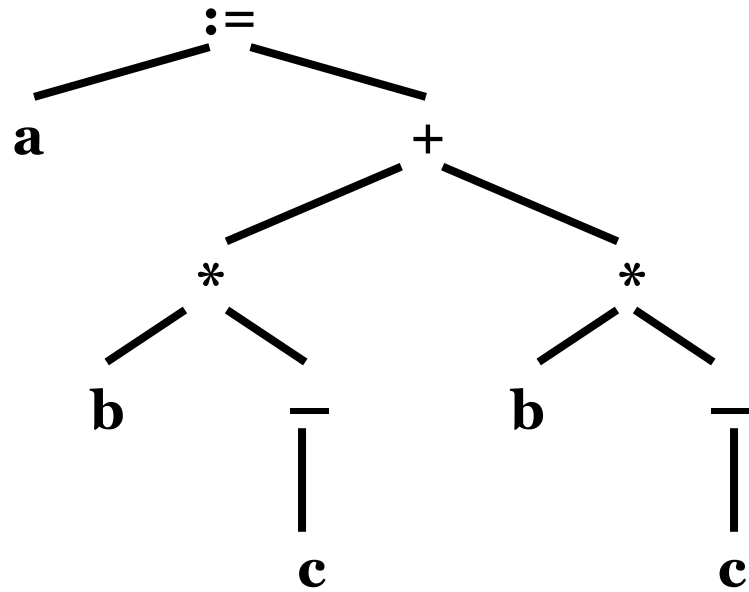
$t1$ y $t2$ Son variables temporales creadas por el compilador.

Esta descomposición de expresiones aritméticas complejas y de proposiciones del flujo de control anidadas hacen al código de 3 direcciones muy adecuado para generar el código objeto y para la optimización. Se llama de 3 direcciones porque cada proposición contiene generalmente ese número de direcciones: 2 para operándos y una para el resultado.

El C3D es también una representación linealizada de un árbol sintáctico.

Ejemplo:

$$a := b * -c + b * -c$$



El C3D seria:

```

t1 := - c
t2 := b * t1
t3 := - c
t4 := b * t3
a := t2 + t4
  
```

Ejercicios: Representar el C3D de las siguientes asignaciones:

```
w := ( a + 5 ) * ( a - 5 ) ;
```

```
q := 5 + 2 * ( a * b ) - a ^ 2 ;
```

```
x := ( - b + √ ( b ^ 2 - 4 * a * c ) ) / ( 2 * a ) ;
```

Tipos de proposiciones de 3 direcciones.

Las proposiciones de 3 direcciones son análogas al ensamblador; las más comunes son:

- a) Asignación de la forma:

$$x := y \text{ op } z$$

- b) Asignación de la forma:

$$x := \text{op } y$$

- c) Proposiciones de copia de la forma:

$$x := y$$

- d) Salto incondicional:

$$\text{goto } E$$

- e) Saltos condicionales, como:

$$\text{if } x \text{ oprel } y \text{ goto } E$$

- f) Llamadas a procedimientos.

$$\begin{array}{c} \text{param } x \\ \text{call } p, m \end{array}$$

Nombre del procedimiento # de argumentos

así como:

$$\text{return } y$$

- g) Asociaciones con índices de la forma:

$$\begin{array}{l} x := y[i] \\ x[i] := y \end{array}$$

- h) Asociaciones de direcciones y apuntadores de la forma

$$\begin{array}{l} x := \& y \\ x := * y \\ * x := y \end{array}$$

3.3 Proposiciones de asignación.

Esquema de traducción para convertir asignaciones a C3D:

```

S → id := E { p := busca ( id.nombre )
               if p ≠ nil then emite ( p := E.Lugar )
               else ERROR }
E → E1 + E2 { E.Lugar := tempnuevo()
               Emite ( E.Lugar := E1.Lugar "+" E2.Lugar ) }
E → - E1 { E.Lugar := tempnuevo()
            Emite ( E.Lugar := "-" E1.Lugar ) }
E → E1 * E2 { E.Lugar := tempnuevo()
               Emite ( E.Lugar := E1.Lugar "*" E2.Lugar ) }
E → ( E1 ) { E.Lugar := E1.Lugar }
E → id { p := busca ( id.nombre )
         if p ≠ nil then E.Lugar := p
         else ERROR }

```

Ejemplo de sentencia válida para la gramática anterior:

valor := x * (a + b * y) + (-z)

Como parte de la traducción de las asignaciones a C3D, se muestra como se pueden buscar los nombres en la tabla de símbolos. El lexema correspondiente al nombre representado como "*id*" viene dado por el atributo "*id.nombre*". La operación "*busca (id.nombre)*" comprueba si existe una entrada para este caso del nombre en la tabla de símbolos, si es así se devuelve el apuntador a la entrada, y en caso contrario "*busca (id.nombre)*" devuelve "*nil*".

Se usa el procedimiento "*emite*" para escribir el C3D a un archivo de salida.

Finalmente la función "*tempnuevo()*" devuelve una secuencia de nombres distintos(*t₁*, *t₂*, *t₃*...) en respuestas a sucesivas llamadas.

Ejemplo: Dada $a := b * -c + b * -c$ generar el C3D equivalente al esquema de traducción anterior:

Esquema de traducción:

```

S → id := E {1}
E → E1 + E2 {2}
E → - E1 {3}
E → E1 * E2 {4}
E → ( E1 ) {5}
E → id {6}

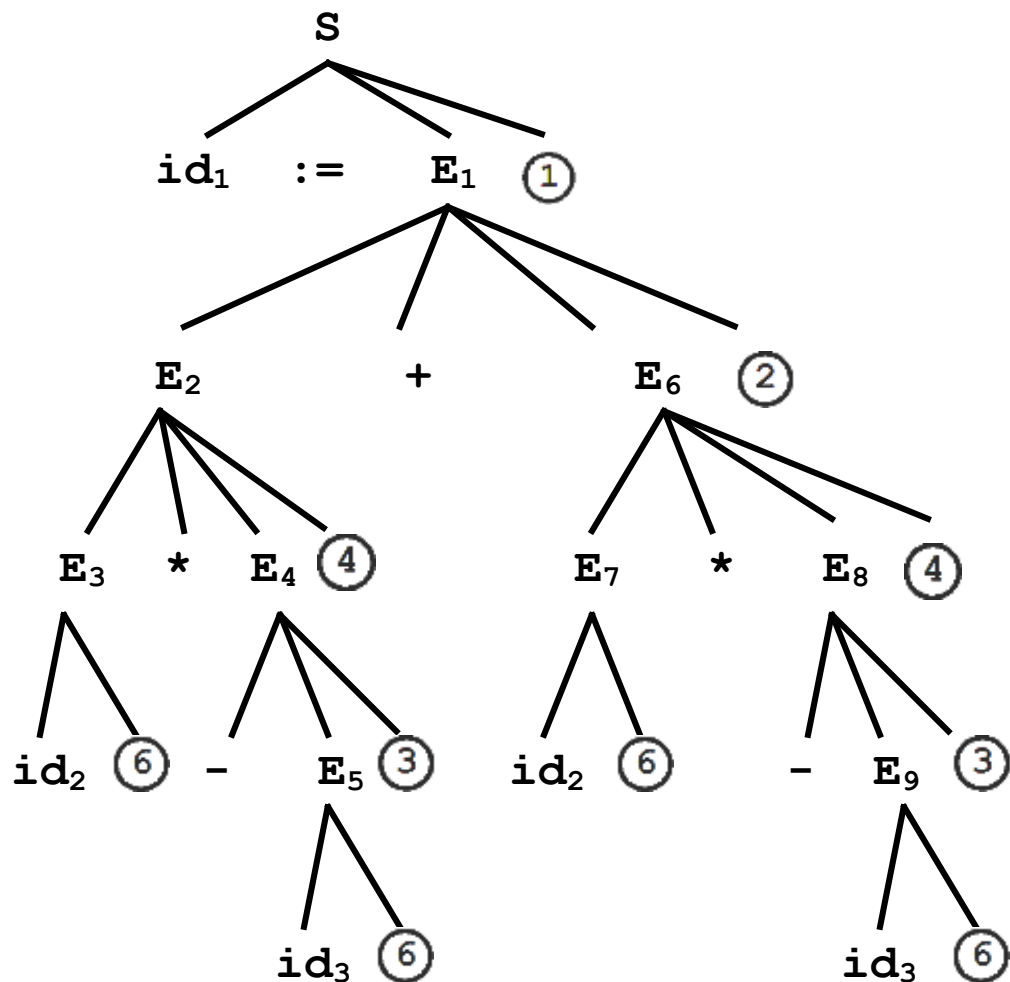
```

1. Análisis Léxico

$id_1 := id_2 * -id_3 + id_2 * -id_3$

TABLA DE SÍMBOLOS

	Lexema	Complex	Tipo	...
1	a	id	...	
2	b	id	...	
3	c	id	...	

2. Análisis Sintáctico3. Generación de Código IntermedioRecorrido en Profundidad.

Símbolo	Lugar
E ₁	t ₅
E ₂	t ₂
E ₃	[2]
E ₄	t ₁
E ₅	[3]
E ₆	t ₄
E ₇	[2]
E ₈	t ₃
E ₉	[3]

**SALIDAS C3D**

```

t1 := -[ 3 ]
t2 := [ 2 ] * t1
t3 := -[ 3 ]
t4 := [ 2 ] * t3
t5 := t2 + t4
[ 1 ] := t5
  
```

3.4 Generación de C3D para proposiciones del flujo del control

$$S \rightarrow \text{if } E \text{ then } S_1 \mid \text{if } E \text{ then } S_1 \text{ else } S_2 \mid \text{while } E \text{ do } S_1$$

En esta gramática anterior el NO-TERMINAL "*E*" se asocia a una expresión booleana y con 2 etiquetas: "*E.verdadera*" y "*E.falsa*". "*E.verdadera*" es la etiqueta a la que fluye el flujo de control si "*E*" es verdadera y "*E.falsa*" es la etiqueta a la que fluye el flujo de control si "*E*" es falsa.

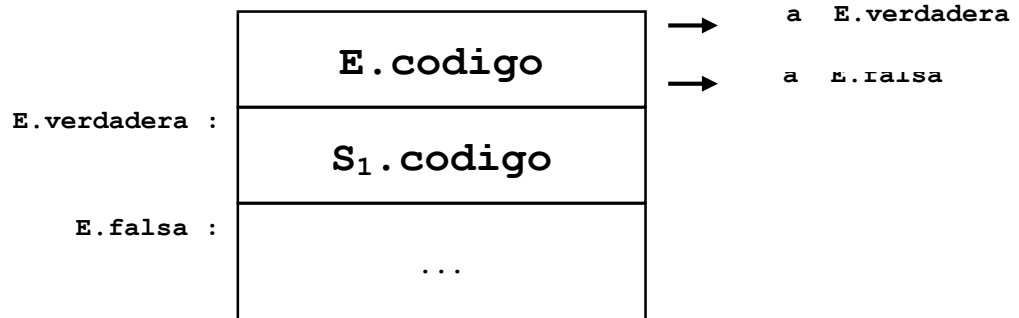
"*S.siguiente*" es una etiqueta que asocia la primera instrucción de 3 direcciones que se ejecuta después del código correspondiente a *S*.

La idea básica de traducción es la siguiente:

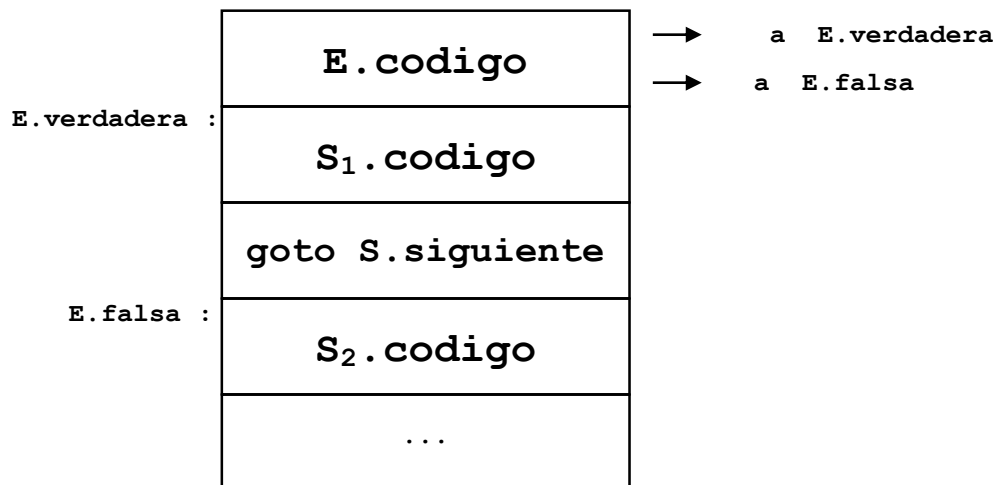
Suponga que "*E*" es de la forma $a < b$ entonces el código generado es de la forma:

```
if a < b goto E.verdadera
goto E.falsa
```

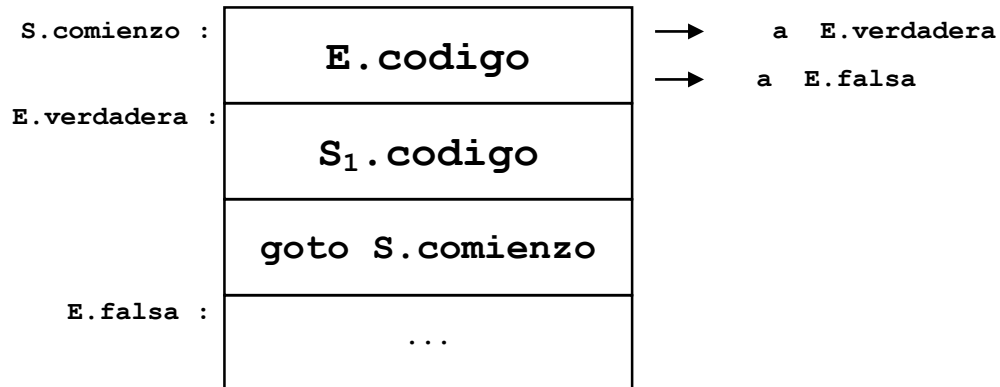
a) if – then



b) if – then - else



c) while – do



3.5 Definiciones dirigidas por la sintaxis para las proposiciones de flujo de control

Producciones:

$S \rightarrow \text{if } E \text{ then } S_1$

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$S \rightarrow \text{while } E \text{ do } S_1$

Reglas semánticas:

E.verdadera:=etiquueva();
E.falsa:= S.siguiente;
S.codigo:= E.codigo||emite(E.verdadera':')
||S₁.codigo;

E.verdadera:=etiquueva();
E.falsa:=etiquueva();
S₁.siguiente:= S.siguiente;
S₂.siguiente:= S.siguiente;
S.codigo:= E.codigo||emite(E.verdadera':')
||S₁.codigo||emite('goto'S.siguiente)
||emite(E.falsa':') ||S₂.codigo;

S.comienzo:=etiquueva();
E.verdadera:=etiquueva();
E.falsa:= S.siguiente;
S₁.siguiente:= S.comienzo;
S.codigo:= emite(S.comienzo':')|| E.codigo
|| emite(E.verdadera':')||S₁.codigo
|| emite('goto' S.comienzo);

Ejemplo: Producir el C3D para el siguiente programa escrito en lenguaje "PROGRA"

```

mientras    x > 10    hacer
inicio
    y:= 1;
fin

```

1. Analisis Léxico

```

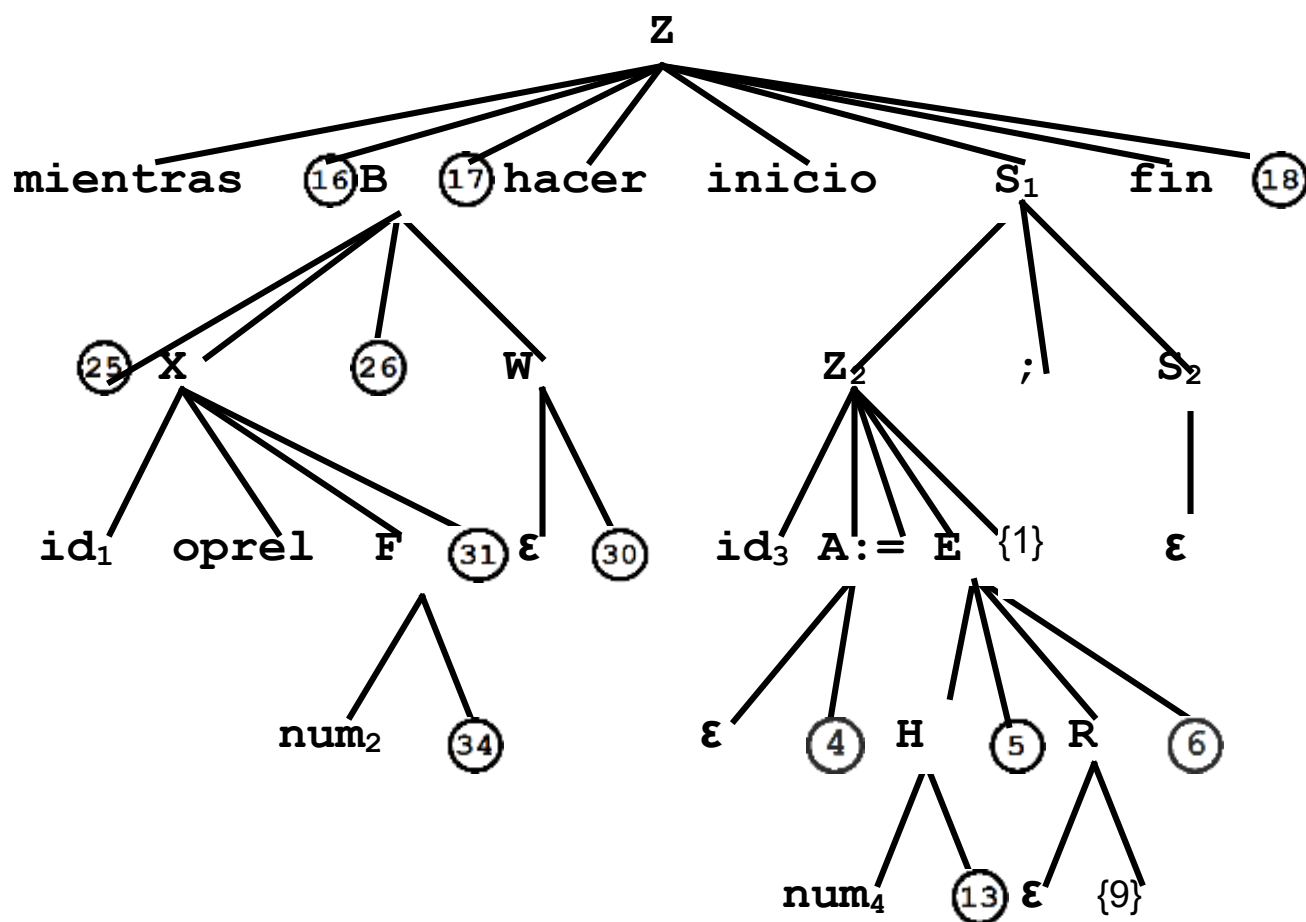
mientras id1 oprel num2 hacer
inicio
    id3:= num4;
fin

```

TABLA DE SÍMBOLOS

	Lexema	Complex	Tipo	...
1	x	id	—	
2	10	num	—	
3	y	id	—	
4	1	num	—	

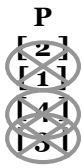
2. Analisis Sintáctico



3. Generación de Código Intermedio

Recorrido de Profundidad.

Símbolo	Lugar	verdadera	falsa	Siguiente	comienzo	h
Z ₁				etiq2	etiq1	
B		etiq3	etiq2			
X		etiq3	etiq2			
F	[2]					
W		etiq3	etiq2			
S ₁		etiq1		etiq1		
Z ₂						
A	<u>nil</u>					
E	[4]					
H	[4]					
R	[4]					[4]
S ₂						



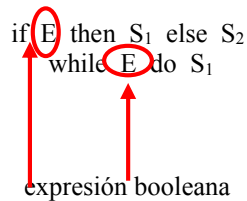
U
[3]

SALIDAS C3D

```

etiq1: if [1]>[2] goto etiq3
        goto etiq2
etiq3:
etiq3: [3]:=[4]
        goto etiq1
etiq2:

```

3.6 Traducción a flujo de control de expresiones booleanas.

Suponga que E es de la forma " E_1 OR E_2 ". Si E_1 es verdadera entonces se sabe inmediatamente que E misma es verdadera, así que $E_1.verdadera$ es lo mismo que $E.verdadera$. Si E_1 es falsa entonces se debe evaluar E_2 , así que se convierte $E_1.falsa$ en etiqueta de la primera proposición en el código de E_2 .

Las salidas con verdadero y falso de E_2 se pueden igualar a las salidas con verdadero y falso de E respectivamente aplicando consideraciones análogas a las traducciones de " E_1 AND E_2 ". Para la expresión de la forma " $NOT E_2$ " solo se intercambian las salidas con verdadero y falso de E_1 para obtenerse las salidas con verdadero y falso de E .

Producciones: **$E \rightarrow E_1 \text{ OR } E_2$** **$E \rightarrow E_1 \text{ AND } E_2$** **$E \rightarrow \text{NOT } E_1$** **$E \rightarrow (E_1)$** **$E \rightarrow id_1 \text{ oprel } id_2$** **$E \rightarrow \text{true}$** **Reglas semánticas:**

```

E1.verdadera:= E.verdadera;
E1.falsa:= etiqnueva();
E2.verdadera:= E.verdadera;
E2.falsa:= E.falsa;
E.codigo:= E1.codigo||emite(E1.falsa':')
           ||E2.codigo;

```

```

E1.verdadera:=etiqnueva();
E1.falsa:= E.falsa;
E2.verdadera:= E.verdadera;
E2.falsa:= E.falsa;
E.codigo:= E1.codigo||emite(E1.verdadera ':')
           ||E2.codigo;

```

```

E1.verdadera:= E.falsa;
E1.falsa:= E.verdadera;
E.codigo:= E1.codigo

```

```

E1.verdadera:= E.verdadera;
E1.falsa:= E.falsa;
E.codigo:= E1.codigo

```

```

E.codigo:= emite('if' id1.Lugar oprel.lexema
                id1.Lugar 'goto' E.verdadera)||
            emite('goto' E.falsa)

```

```

E.codigo:= emite('goto' E.verdadera)
            emite('goto' E.falsa)

```

Ejemplo: Generar el C3D del siguiente código en lenguaje PROGRA usando el Esquema de Traducción de la página 49:

```

Si a >= 5 Y a <= 10 entonces
    inicio
        a := 0;
    fin
Sino
    inicio
        a := 100;
    fin

```

4. Análisis Léxico

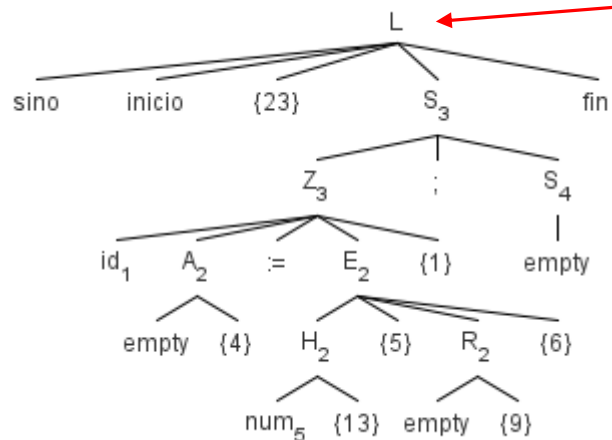
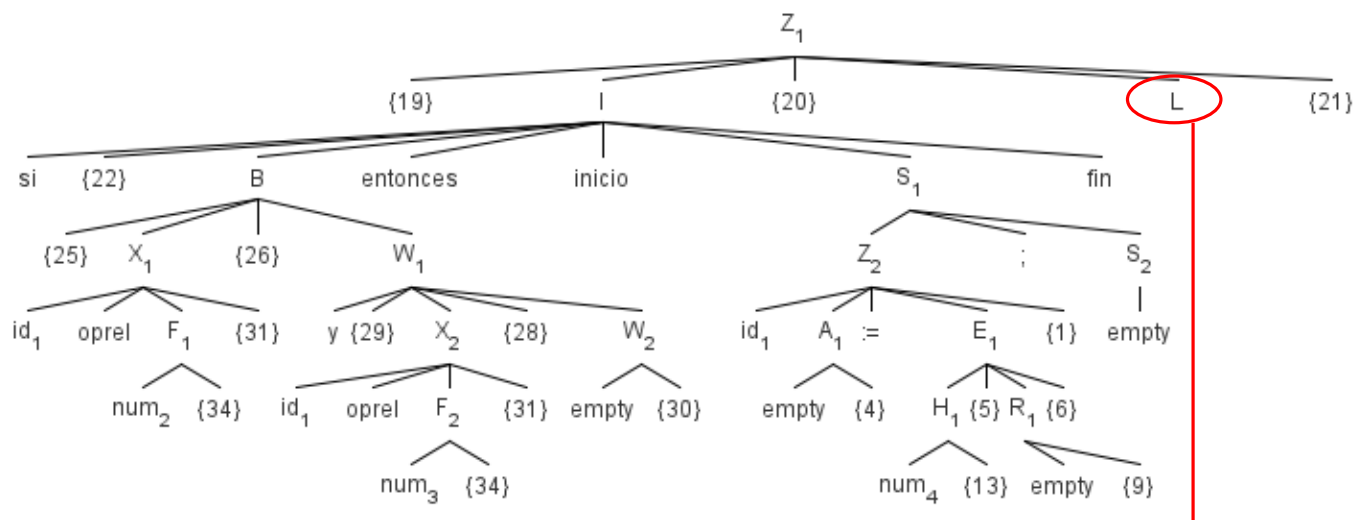
```

si id1 oprel num2 Y id1 oprel num3 entonces
  inicio
    id1 := num4;
  fin
sino
  inicio
    id1 := num5;
  fin

```

TABLA DE SÍMBOLOS

	Lexema	Complex	Tipo	...
1	a	id	...	
2	5	num	...	
3	10	num	...	
4	0	num	...	
5	100	num	...	

5. Análisis Sintáctico

6. Generación de Código IntermedioRecorrido de Profundidad.

Símbolo	Lugar	verdadera	falsa	siguiente	comienzo	h
Z ₁				etiq1		
I				etiq2		
B		etiq3	etiq2			
X ₁		etiq3	etiq2			
W ₁		etiq4	etiq2			
F ₁	[2]					
X ₂		etiq3	etiq2			
W ₂			etiq2			
F ₂	[3]					
S ₁				etiq2		
S ₂						
Z ₂						
A ₁	nil					
E ₁	[4]					
H ₁	[4]					
R ₁	[4]					[4]
L				etiq1		etiq2
S ₃						
Z ₃						
A ₂	nil					
E ₂	[5]					
H ₂	[5]					
R ₂	[5]					[5]
S ₄						

**SALIDAS C3D**

```

if [1]>=[2] goto etiq3
goto etiq2
etiq3:
if [1]<=[3] goto etiq4
goto etiq2
etiq4 :
[1]:=[4]
goto etiq1
etiq2:
[1]:=[5]
etiq1:

```

EJERCICIO. Implementar en Java el siguiente Esquema de Traducción que genera C3D para lenguaje Simple.

$P \rightarrow V C$

$V \rightarrow \text{id} : T \quad V_1 \mid \text{empty}$

$T \rightarrow \text{entero} \mid \text{real} \mid \text{character}$

$C \rightarrow \text{inicio S fin}$

$S \rightarrow \text{id opasig E} \{1\} \mid S \mid S$

$E \rightarrow \text{num E}' \{5\} \mid \text{num.num E}' \{6\} \mid \text{id E}' \{7\}$

$E' \rightarrow \text{oparit E} \{8\} \mid \text{empty} \{9\}$

No. Accion	Accion Semántica
1	<pre>p := busca (id.lexema) if p != NIL then emite (p ':' E.Lugar) else ERROR</pre>
5	<pre>if E'.op == "" then E.Lugar := num.entrada else begin E.lugar := tempnuevo () emite (E.Lugar ':' num.entrada E'.op E'.Lugar) end</pre>
6	<pre>if E'.op == "" then E.Lugar := num.num.entrada else begin E.lugar := tempnuevo () emite (E.Lugar ':' num.num.entrada E'.op E'.Lugar) end</pre>
7	<pre>if E'.op == "" then E.Lugar := id.entrada else begin E.lugar := tempnuevo () emite (E.Lugar ':' id.entrada E'.op E'.Lugar) end</pre>
8	<pre>E'.op := oparit.lexema E'.Lugar := E.Lugar</pre>
9	<pre>E'.op := ""</pre>

3.7 Caso de Estudio

Generador de Código Intermedio para Lenguaje PROGRA

```

P -> D C
D -> V ; D | ε
V -> id : T
T -> caracter | entero | real | &T | arreglo [ num ] of T
C -> inicio S fin
S -> Z ; S | ε
Z -> id A := E {1}
Z -> {19} I {20} L {21}
I -> si {22} B entonces inicio S fin
L -> sino inicio {23} S fin | ε {24}
Z -> mientras {16} B {17} hacer inicio S fin {18}
E -> H {5} R {6}
R -> oparit H {7} R {8} | ε {9}
H -> ( E ) {10} | -E {11} | id A {12} | num {13} | num.num {14} | literal {15}
A -> [ E ] {2} | & {3} | ε {4}
B -> {25} X {26} W
W -> O {27} X {28} W | Y {29} X {28} W | ε {30}
X -> ( {33} X ) | id oprel F {31} | cierto {32} | falso
F -> id A {37} | num {34} | num.num {35} | literal {36}

```

TABLA DE ACCIONES SEMANTICAS

Acción Semántica	Código de la Acción Semántica	# Error del GCI
1	<pre> p := id.entrada; if p <> nil then begin u := if A.lugar = nil then p else p ', ' A.lugar; emite (u ':' E.lugar); end else error; </pre>	101
2	A.lugar := E.lugar;	
3	A.lugar := nil;	
4	A.lugar := nil;	
5	R.h := H.lugar;	
6	E.lugar := R.lugar;	
7	<pre> R₁.h := tempnuevo; emite (R₁.h ':' R.h oparit.lexema H.lugar); </pre>	
8	R.lugar := R ₁ .lugar;	
9	R.lugar := R.h;	
10	H.lugar := E.lugar;	
11	<pre> H.lugar := tempnuevo; emite (H.lugar ':' menosu' E.lugar); </pre>	
12	<pre> p := id.entrada; if p <> nil then begin u := if A.lugar = nil then p else p ', ' A.lugar; H.lugar := u; end else error; </pre>	102
13	p := num.entrada;	

	if p <> nil then H.lugar := p else error;	103
14	p := num.num .entrada; if p <> nil then H.lugar := p else error;	104
15	p := literal .entrada; if p <> nil then H.lugar := p else error;	105
16	Z.comienzo := etiqnueva; Z.siguiente := etiqnueva; B.verdadera := etiqnueva; B.falsa := Z.siguiente; S.siguiente := Z.comienzo; emite (Z.comienzo ':');	
17	emite (B.verdadera ':');	
18	emite (' goto ' Z.comienzo); emite (B.falsa ':');	
19	Z.siguiente := etiqnueva; I.siguiente := etiqnueva;	
20	L.h := I.siguiente; L.siguiente := Z.siguiente;	
21	emite (L.siguiente ':');	
22	B.verdadero := etiqnueva; B.falsa := I.siguiente; S.siguiente := I.siguiente;	
23	emite (' goto ' L.siguiente); emite (L.h ':');	
24	L.siguiente := L.h;	
25	X.verdadero := B.verdadero; X.falso := B.falsa;	
26	W.verdadero := X.verdadero; W.falsa := X.falsa;	
27	X.verdadero := W.verdadero; X.falsa := W.falsa; W.falsa := etiqnueva; emite (' goto ' W.falsa); emite (W.falsa ':');	
28	W ₁ .verdadero := X.verdadero; W ₁ .falsa := X.falsa;	
29	X.verdadero := W.verdadero; X.falsa := W.falsa; W.verdadero := etiqnueva; emite (' goto ' X.falsa); emite (X.verdadero ':'); X.verdadero := W.verdadero;	
30	emite (' goto ' W.falsa); emite (W.verdadera ':');	
31	p := id .entrada; if p <> nil then emite (' if ' p oprel.lexema F.lugar ' goto ' X.verdadera) else error;	106
32	emite (' goto ' X.verdadera);	
33	X ₁ .verdadera := X.verdadera; X ₁ .falsa := X.falsa;	
34	p := num .entrada; if p <> nil then F.lugar := p else error;	107
35	p := num.num .entrada; if p <> nil then F.lugar := p else error;	108
36	p := literal .entrada; if p <> nil then F.lugar := p else error;	109
37	p := id .entrada; if p <> nil then begin u := if A.lugar = nil then p else p ',' A.lugar; F.lugar := u; end	

	<pre>else error;</pre>	110
--	--------------------------	-----